

Three parts. Reports over the catalogue that need arrays, a join and buckets; one query made fast with the right index, proven with explain; and a validator for the islands of A01, followed by a change of shape under that validator. Everything was typed in the sessions *Aggregation I*, *Aggregation II* and *Indexes and Schema Validation*.

Submission Requirements:

- Read the *Organizational Guidelines* on Moodle, and adhere to them
 - Submit a PDF with, per task, your solution idea, the code, and the output. Name and time spent on the cover page
 - Submit a ZIP with your files. mongosh scripts (.js) or PHP scripts are both fine; a mongosh script must run with `docker compose exec -T mongodb mongosh -quiet catalogue < file.js`, a PHP script from app/a02/ as in A01
 - Naming: MDB3_A02_LASTNAME.pdf and MDB3_A02_LASTNAME.zip
 - Do not modify the two catalogue collections. Indexes you create for task 2 are dropped again at the end of your script; task 3 works in a database of your own
-

Exercise 1 (12 points) Three Reports

Each report is one aggregation pipeline on catalogue. For each: the pipeline, its output, and one sentence saying what the SQL would have been (which tables, which JOIN, which GROUP BY).

1. **Rich species (4 points).** For every species: the total sell value of all furniture its villagers own, and how many ownerships that is. The five richest species, richest first. (Ownerships are the furniture arrays; prices are in items.)
2. **Materials (4 points).** For every recipe material: how many recipes use it, and the name of one item that is crafted with it. The five most used materials. (Recipes are embedded in the items that have one, as `recipe.materials`.)
3. **Collectors (4 points).** Villagers grouped by how many items they own: 0 to 9, 10 to 14, 15 to 19, 20 and more. Per group the count and three example names. One stage does the grouping.

Exercise 2 (4 points) explain, Before and After

The query: all villagers born in October, sorted by name.

```
db.villagers.find({ "birthday.month": 10 }).sort({ name: 1 })
```

1. Run it with `explain("executionStats")` and note `nReturned`, `totalKeysExamined`, `totalDocsExamined` and the stage names, innermost to outermost
2. Create the one index that serves both the filter and the sort. Run the explain again and note the same four things
3. In two sentences: why does `{name: 1, "birthday.month": 1}` not do the same job? (Try it if you like.)
4. Drop your indexes at the end

Exercise 3 (8 points) A Validator for the Islands, and a New Shape

Work in a database of your own (a02). Two starting points, graded the same:

- **Your own islands.** Insert three or four island documents in the shape of your A01 solution (a script with one `insertMany` is enough; you do not need your DAO). The validator and the migration then follow **your** shape.
- **The reference islands.** The starter on Moodle (A02_starter.zip) contains one script, `seed_islands.mongosh.js`. It creates `a02.islands` with four islands in the reference shape below. Use it if your A01 model differs from the reference or if you want a clean start.

```
// the reference shape from A01 (version 1); nativeFruit is optional
{ _id: ObjectId, name: string, player: string, hemisphere: "north" | "south",
  nativeFruit: string, created: Date,
  residents: [ { villagerId: int, name: string } ],           // at most ten
  inventory: [ { itemId: int, name: string, count: int } ] }  // count >= 1
```

1. **The validator (4 points).** Write a `$jsonSchema` validator for islands that enforces what A01 described: the required fields, hemisphere as one of two values, created as a date, residents as an array of at most ten `{villagerId, name}` documents with the right types, inventory entries with an integer count of at least 1. Show one insert that is accepted and one that is refused for at least two different reasons, with the error's list of failing fields.

2. **Version 2 (4 points)**. Two changes, and every document carries `schemaVersion: 2`:
- The native fruit becomes a list: `nativeFruit` (an optional string) is replaced by `fruits` (an array of strings, possibly empty).
 - A **new required field** `status`, one of "active" or "archived". The existing islands do not have it, so the migration has to **backfill** it by a rule: an island where nobody lives and nothing is stored is "archived", every other island is "active".
- Do it in the order from the lecture and show each step: widen the validator so that v1 and v2 documents are both accepted; migrate every document with **one** `updateMany` (an absent `nativeFruit` becomes an empty array, `status` follows the rule); narrow the validator to v2 only and prove with three inserts that a v1 document is now refused, a v2 document without `status` is refused, and a complete v2 document is accepted.